بِسْمِ اللّٰهِ الرَّحْمٰنِ الرَّحِيْمِ

اَلْحَمْدُ لِلّٰهِ رَبِّ الْعَالَمِيْنَ

وَالصَّلٰوةُ وَالسَّلَامُ عَلٰى خَاتَمِ النَّبِيِّيْنَ

Buffer Overflow

Computer Security:Principles and Practice

Fourth Edition By: William Stallings and Lawrie Brown

Chapter 10

A Brief History of Some Buffer Overflow Attacks

1988	The Morris Internet Worm uses a buffer overflow exploit in "fingerd" as one of its attack mechanisms.
1995	A buffer overflow in NCSA httpd 1.3 was discovered and published on the Bugtraq mailing list by Thomas Lopatic.
1996	Aleph One published "Smashing the Stack for Fun and Profit" in *Phrack* magazine, giving a step by step introduction to exploiting stack-based buffer overflow vulnerabilities.
2001	The Code Red worm exploits a buffer overflow in Microsoft IIS 5.0.
2003	The Slammer worm exploits a buffer overflow in Microsoft SQL Server 2000.
2004	The Sasser worm exploits a buffer overflow in Microsoft Windows 2000/XP Local Security Authority Subsystem Service (LSASS).

Buffer Overflow

A condition at an interface under which more input can be placed into a buffer or data holding area than the capacity allocated, overwriting other information. Attackers exploit such a condition to crash a system or to insert specially crafted code that allows them to gain control of the system.

Safe unsafe languages

What is the difference between statically and dynamically typed languages?

Java and python: Safe but minimal system (hardware and memory) control

Assembly: Unsafe but maximum system (hardware and memory) control

C: In Between both hence used for most operating systems and device drivers

Unsafe input methods: without any checks, but exist for efficiency in C. Not in high level languages like java and python

Buffer Overflow

- A very common attack mechanism
  - First widely used by the Morris Worm in 1988
- Prevention techniques known
- Still of major concern
  - **Legacy of buggy code in widely deployed operating systems and applications**
  - **Continued careless programming practices by programmers**

Buffer Overflow Basics

- Programming error when a process attempts to store data beyond the limits of a fixed-sized buffer
- Overwrites adjacent memory locations
  - **Locations could hold other program variables, parameters, or program control flow data**
- Buffer could be located on the stack, in the heap, or in the data section of the process

Consequences:

- **Corruption of program data**
- **Unexpected transfer of control**
- **Memory access violations**
- **Execution of code chosen by attacker**

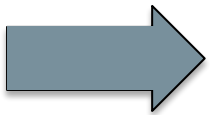
Programming Language History

- At the machine level data manipulated by machine instructions executed by the computer processor are stored in either the processor's registers or in memory

- Assembly language programmer is responsible for the correct interpretation of any saved data value

Modern high-level languages have a strong notion of type and valid operations

- **Not vulnerable to buffer overflows**
- **Does incur overhead, some limits on use**



C and related languages have high-level control structures, but allow direct access to memory

- **Hence are vulnerable to buffer overflow**
- **Have a large legacy of widely used, unsafe, and hence vulnerable code**

Some Common Unsafe C Standard Library Routines

`gets(char *str)`	read line from standard input into str
`sprintf(char *str, char *format, ...)`	create str according to supplied format and variables
`strcat(char *dest, char *src)`	append contents of string src to string dest
`strcpy(char *dest, char *src)`	copy contents of string src to string dest
`vsprintf(char *str, char *fmt, va_list ap)`	create str according to supplied format and variables

Note that these do not even have to be in the code for a particular program, they can (and indeed do) occur in library routines used by programs, including both standard libraries and third-party application libraries.

C Safer alternatives

Safer alternatives mention the max size i.e. number of of CHARS to read/copy

Cost: Computational overhead

Size of a CHAR = 1 Byte = 8 bits

i.e. enough for ASCII characters

Not for unicode e.g. UTF16/32 need 2 / 4 CHARS

Unsafe Function	Safer Alternative
gets	fgets
sprintf	snprintf
strcat	strncat
strcpy	strncpy
vsprintf	vsnprintf

Basic Buffer Overflow C Code

```
int main(int argc, char *argv[]) {
   int valid = FALSE;
   char str1[8];
   char str2[8];
   strcpy(str1, "START");                    #Unsafely copies a string “START” str2.
   gets(str2);                               #Unsafely inputs a line of string input in str2
   if (strncmp(str1, str2, 8) == 0)          # Compares both strings upto n i.e. 8 characters
       valid = TRUE;
   printf("buffer1: str1(%s), str2(%s), valid(%d)\n", str1, str2, valid);
}
```

*Three variables (valid , str1, and str2), whose order and location in memory will depend on the variable type (**local** or **global**), the **language**, **compiler** used, and the target **machine architecture**. But will **typically** saved in adjacent memory locations for local variables in a C function on common processor architectures such as the Intel Pentium family.*

Str1 and str2 expecting 7 bytes at max with a null terminator at end

Basic Buffer Overflow Example Run

$ cc -g -o buffer1 buffer1.c

$ ./buffer1

START

buffer1: str1(START), str2(START), valid(1)

$ ./buffer1

EVILINPUTVALUE

buffer1: str1(TVALUE), str2(EVILINPUTVALUE), valid(0)

$ ./buffer1

BADINPUTBADINPUT

buffer1: str1(BADINPUT), str2(BADINPUTBADINPUT), valid(1)

Memory Address	Before gets(str2)	After gets(str2)	Contains Value of
. . . .	. . . .	. . . .	
bffffbf4	34fcffbf 4 . . .	34fcffbf 3 . . .	argv
bffffbf0	01000000	01000000	argc
bffffbec	c6bd0340 . . . @	c6bd0340 . . . @	return addr
bffffbe8	08fcffbf	08fcffbf	old base ptr
bffffbe4	00000000	01000000	valid
bffffbe0	80640140 . d . @	00640140 . d . @	
bffffbdc	54001540 T . . @	4e505554 N P U T	str1[4-7]
bffffbd8	53544152 S T A R	42414449 B A D I	str1[0-3]
bffffbd4	00850408	4e505554 N P U T	str2[4-7]
bffffbd0	30561540 0 V . @	42414449 B A D I	str2[0-3]
. . . .	. . . .	. . . .	

Memory Address	Before gets(str2)	After gets(str2)	Contains Value of
. . . .	. . . .	. . . .	
bffffbf4	34fcffbf 4 . . .	34fcffbf 3 . . .	argv
bffffbf0	01000000	01000000	argc
bffffbec	c6bd0340 . . . @	c6bd0340 . . . @	return addr
bffffbe8	08fcffbf	08fcffbf	old base ptr
bffffbe4	00000000	01000000	valid
bffffbe0	80640140 . d . @	00640140 . d . @	
bffffbdc	54001540 T . . @	4e505554 N P U T	str1[4-7]
bffffbd8	53544152 S T A R	42414449 B A D I	str1[0-3]
bffffbd4	00850408	4e505554 N P U T	str2[4-7]
bffffbd0	30561540 0 V . @	42414449 B A D I	str2[0-3]
. . . .	. . . .	. . . .	

Figure 10.2 Basic Buffer Overflow Stack Values

Assuming that the variables are saved in consecutive memory locations, from highest to lowest

This will typically be the case for local variables in a C function on common processor architectures such as the Intel Pentium family.

E.g. Attacker can bypass an authentication if the code was meant to compare a password input in str2 with the password stored in str1

By inputting **BADINPUTBADINPUT** (16 characters) instead of 7 that can be stored in buffer of str2 (alongwith terminating NULL)

Buffer Overflow Attacks

- To exploit a buffer overflow an attacker needs:
  - To identify a buffer overflow vulnerability in some program that can be triggered using externally sourced data under the attacker's control
  - To understand how that buffer is stored in memory and determine potential for corruption
- Identifying vulnerable programs can be done by:
  - Inspection of program source
  - Tracing the execution of programs as they process oversized input
  - Using tools such as fuzzing to automatically identify potentially vulnerable programs

Stack Overflows

- Occur when buffer is located on stack
  - Also referred to as *stack smashing*
  - Used by Morris Worm
  - Exploits included an unchecked buffer overflow
- Are still being widely exploited
- Stack frame
  - When one function calls another it needs somewhere to save the return address
  - Also needs locations to save the parameters to be passed in to the called function and to possibly save register values

Function Stack

Stack Memory used to store the contents for the function variables

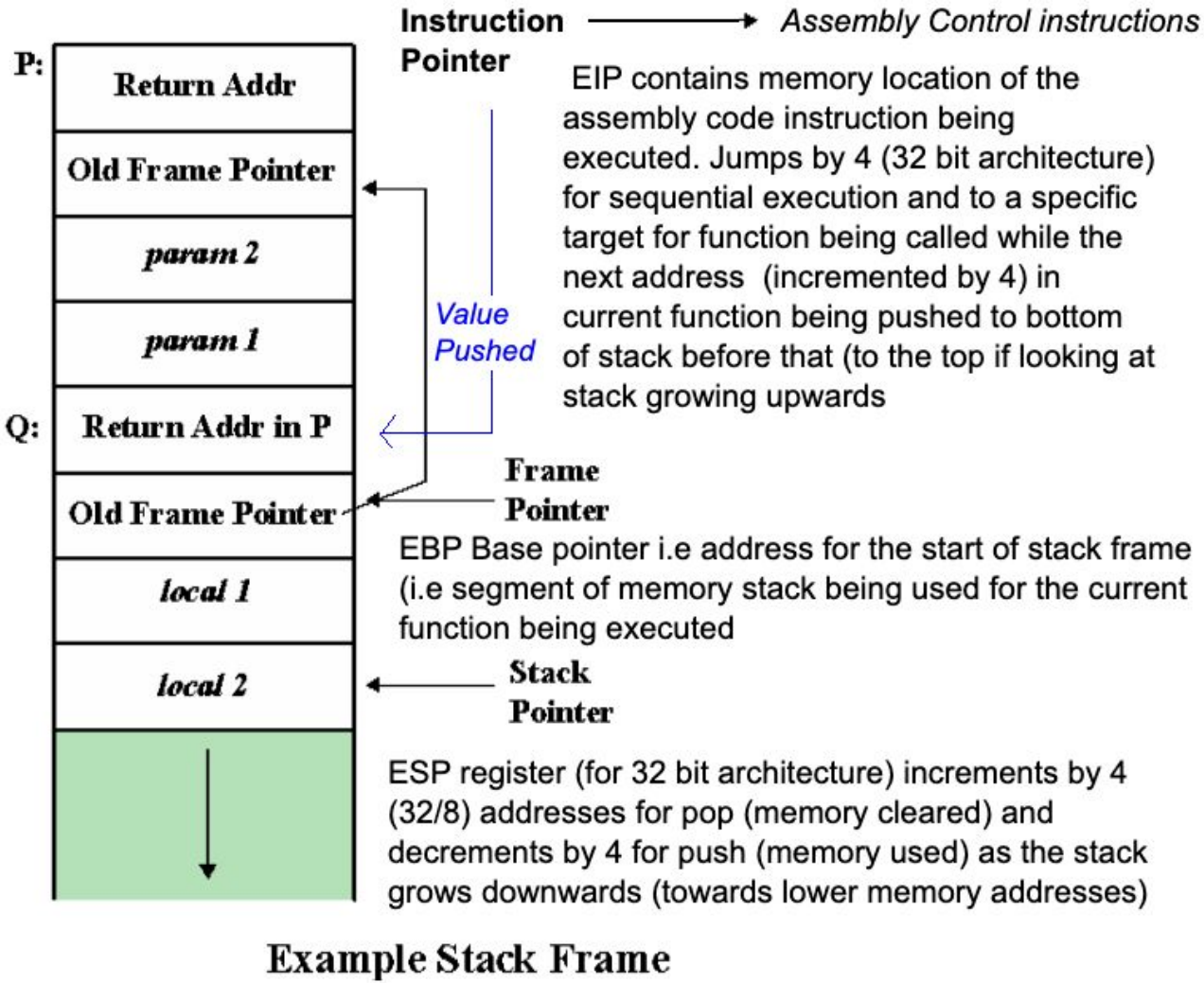
Besides the CPU registers

EIP (Instruction)

ESP (Stack)

EBP (Base of stack)

EAX (Accumulation)



Code Compiler Revised

```
int add(int a, int b) {
    return a + b;
}
int main() {
    int result = add(3, 4);
    return result;
}
```

```
Main:
push 4          ; put b on stack
push 3          ; put a on stack
call add        ; push return address
                    + jump to add
```

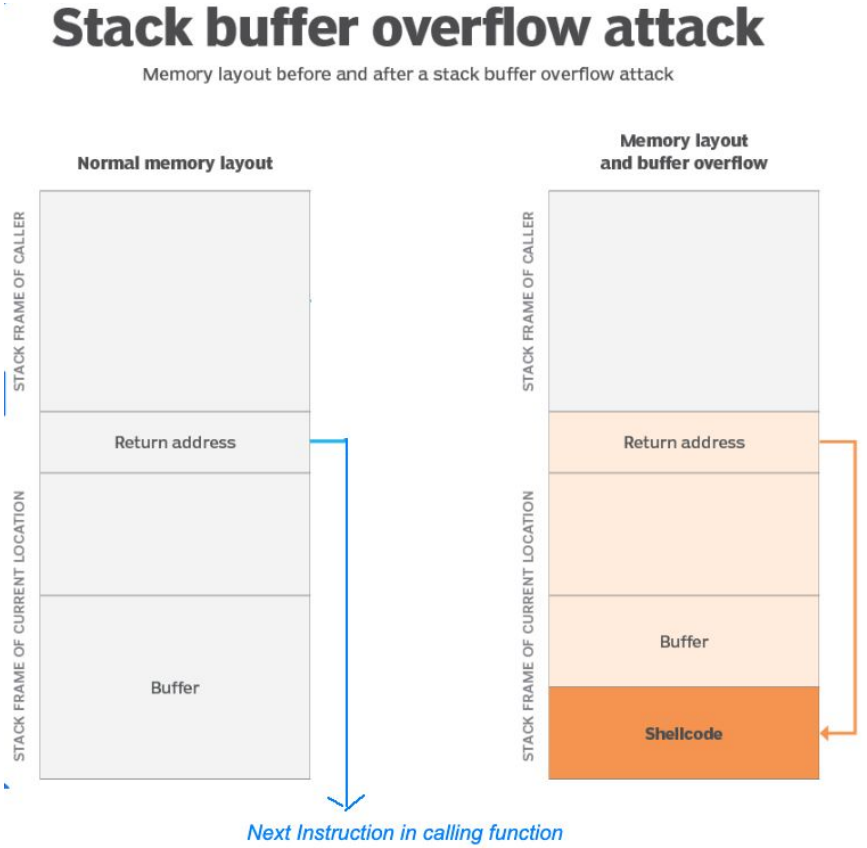
```
add:
    push ebp          ; save old base pointer
    mov ebp, esp      ; start new stack frame
    mov eax, [ebp+8]  ; load a (3)
    mov edx, [ebp+12] ; load b (4)
    add eax, edx            ; eax = a+b = 7
    pop ebp           ; restore old base pointer
    ret               ; pop return address
                      + jump back to main
```

Stack overflow exploit

When a function is called its passed a memory space for variables to fill, followed by a return address for memory location of next instructions in code

If the space for variables overflows, return address can be overwritten

1. Resulting in **segment fault** if overwritten value is for **illegal address**, hence **denial of service**
2. Can be **exploited** to return to the address for **same** variable written with **malicious instruction**
3. Or to return to another location where malicious machine instructions (**shell code**) are placed beforehand



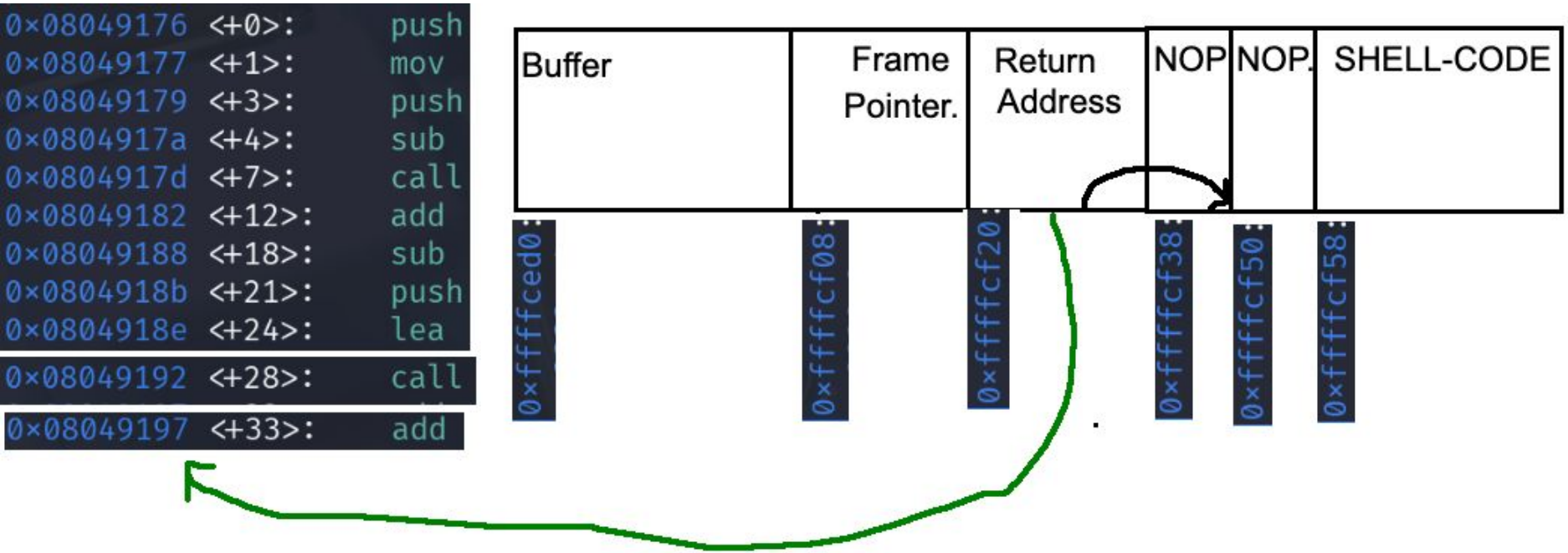
https://www.techtarget.com/whatis/definition/stack-overflow

Shellcode

- Code supplied by attacker
  - Often saved in buffer being overflowed
  - Traditionally transferred control to a user command-line interpreter (shell)
- Machine code: can directly run on CPU
  - **Named shell code due to mainly used objective of obtaining target shell**
  - **Specific to processor and operating system**
  - Traditionally needed good assembly language skills to create
  - More recently a number of sites and tools have been developed that automate this process

https://www.exploit-db.com/shellcodes

Stack Overflow Exploit Example



Other Shellcode functions

Launch a remote shell when connected to

Create a reverse shell that connects back to the hacker

Use local exploits that establish a shell

Flush firewall rules that currently block other attacks

Break out of a chroot (restricted execution) environment, giving full access to the system

Metasploit Framework (MSF)

Open-source command-line tool used throughout the penetration testing/hacking

msfconsole the central command-line interface where you run everything.

Metasploit Pro: version with a GUI but that is not free

MSF Modules: components for:

- Exploits – trigger a vulnerability
- Scanners / Auxiliary – gather info or test services
- Payloads – what runs on the target once exploited

Used to create input patterns (msf-pattern_create) to test/trigger vulnerabilities including stack overflow

https://tryhackme.com/room/metasploitintro

Kali Linux

A Linux distribution tailored for cybersecurity and penetration testing.

Preloaded with hundreds of hacking/pen-testing tools including Metasploit

Provides easy system control e.g. to:

- Disable Address Space Randomization Layout ASLR
- Spoofing MAC/IP address

Virtualbox (or any vm player)

https://www.virtualbox.org/wiki/Downloads

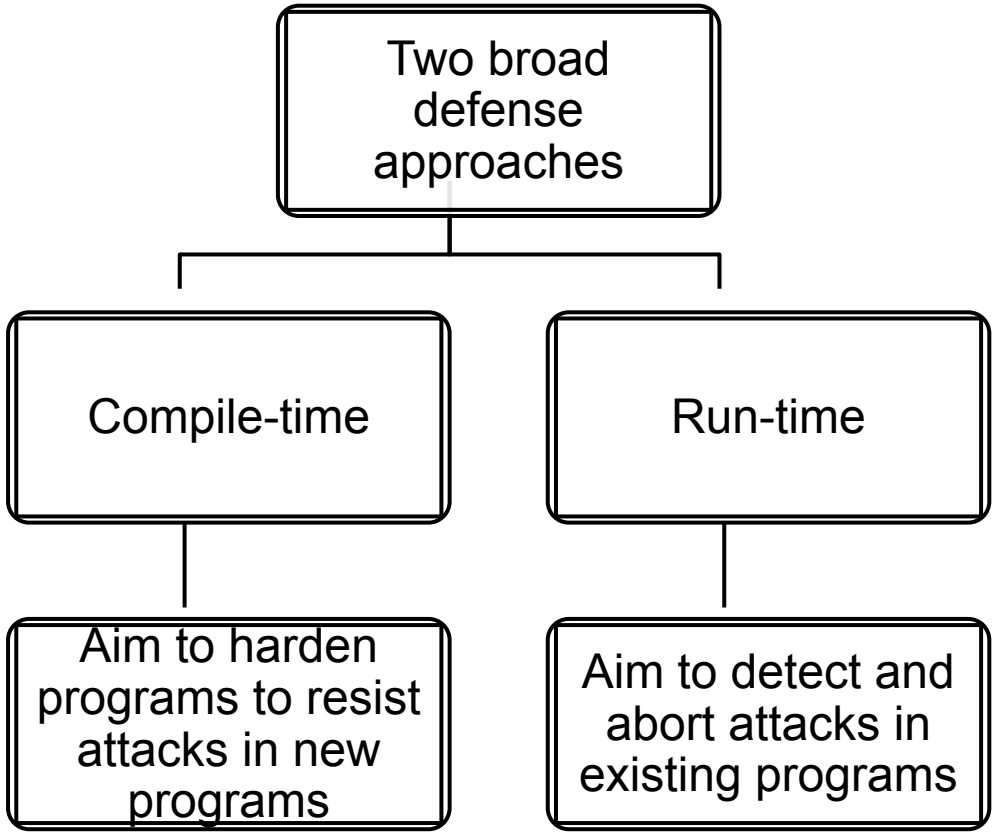
Kali linux VM

https://www.kali.org/get-kali/#kali-virtual-machines Defaults: kali/kali

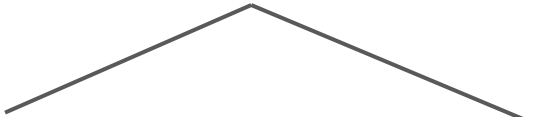
Buffer Overflow Defenses

Compile time defence:
needs to re-compile the code
hence for **new Programmes**

Run-Time defence: without recompiling, hence useful for **existing programmes**



Defense against buffer overflow



Compile-time Defense

- Safe programming Language
- Safe coding
- Safe Libraries
- Stack Protection *(copying return address value to redundant safe locations)*

Run-time Defense

1. Non Executable regions
2. Address Space Randomization
3. Guard Pages (illegal addresses)

Compile-Time Defenses: Programming Language

- Use a modern high-level language (e.g. **java, python**)
  - Not vulnerable to buffer overflow attacks
  - Compiler enforces range checks and permissible operations on variables

Disadvantages

- Additional code must be executed at run time to impose checks
- Flexibility and safety comes at a cost in **resource use**
- Distance from the underlying machine language and architecture means that access to some instructions and hardware resources is lost
- Limits their **usefulness** in writing code, such as device drivers, that must interact with such resources

Compile-Time Defenses: Safe Coding Techniques

- C designers placed much more emphasis on space **efficiency** and performance considerations than on type safety
  - Assumed programmers would exercise due care in writing code
- Programmers need to inspect the code and rewrite any unsafe coding
  - An example of this is the OpenBSD project
- Programmers have **audited** the existing code base, including the operating system, standard libraries, and common utilities
  - This has resulted in what is widely regarded as one of the safest operating systems in widespread use

Compile-Time Defenses:

Language Extensions/Safe Libraries

- Concern with C is use of unsafe **standard** library routines
  - One approach has been to replace these with **safer variants**
    - Libsafe is an example
    - Library is implemented as a dynamic library arranged to load before the existing standard libraries

Compile-Time Defenses: Stack Protection

- Add function entry and exit code to check stack for signs of corruption
- **Stackshield** and **Return Address Defender (RAD)**
  - **GCC extensions** that include additional function entry and exit code
    - **Function entry writes a copy of the return address to a safe region of memory**
    - **Function exit code checks the return address in the stack frame against the saved copy**
    - If change is found, aborts the program

Run-Time Defenses: Executable Address Space Protection

Use virtual memory support to make some regions of memory non-executable

Issues

- **Requires support from memory management unit (MMU)**
- **Long existed on SPARC / Solaris systems**
- **Recent on x86 Linux/Unix/Windows systems**

Run-Time Defenses: Address Space Randomization

- Manipulate location of key data structures
  - Stack, heap, global data
  - Using random shift for each process
  - Large address range on modern systems means wasting some has negligible impact
- Randomize location of heap buffers
- Random location of standard library functions

Run-Time Defenses: Guard Pages

- Place guard pages between critical regions of memory
  - **Flagged in MMU as illegal addresses**
  - Any attempted access aborts process
- Further extension places guard pages Between stack frames and heap buffers
  - Cost in execution time to support the large number of page mappings necessary